밑바닥부터 만드는 Redis: 시니어와 주니어의 아키텍처 티키타카

Python 내장 라이브러리 0%, 순수 포인터와 배열만으로 O(1) 캐시 서버 구축하기



Gemini Notebook

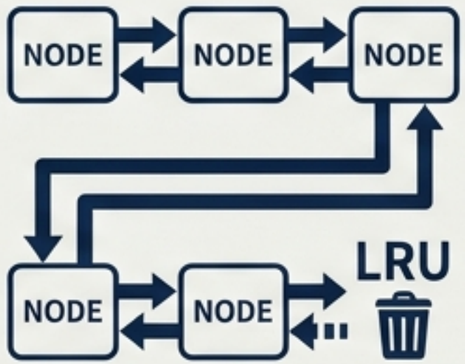
편안한 마법을 버리고 험난한 엔지니어링을 택하다





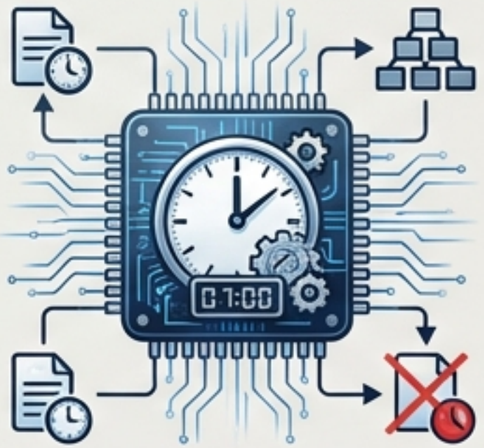
체이닝 해시맵
(Chaining HashMap)

O(1) 조회와 동적 리사이징



이중 연결 리스트
(Doubly Linked List)

O(1) LRU 메모리 방출



최소 힙
(Min-Heap)

힙 기반 O(1) TTL 만료
시스템 및 지연 삭제

Gemini Notebook

시니어 아키텍트 '알렉스'

시간/공간 복잡도와 포인터 연산의 원자성에 타협은 없습니다.

내장 라이브러리의 안락함 뒤에 숨은 비용에 엄격한 베테랑.



3개월 차 주니어 '제이'

그래서 이게 일상생활로 치면 뭐랑 같은 건가요?

복잡한 알고리즘을 찰떡같은 비유로 번역하는 천재.

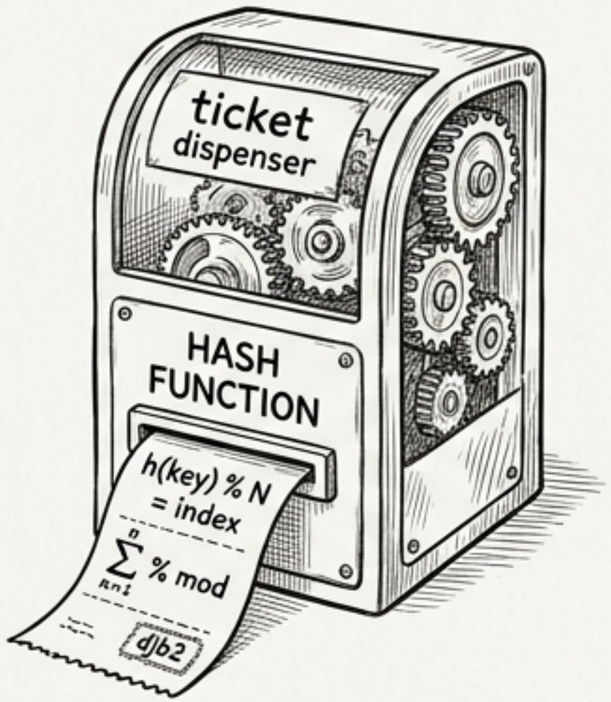


Gemini Notebook

토론 1. 내장 dict를 놔두고 왜 체이닝 해시맵을 손수 만들었는가?

제이, 파이썬의 dict는
훌륭하지만 내부 원리를
모르면 블랙박스일 뿐입니다.

해시 충돌과 동적 리사이징을
밑바닥부터 제어해봐야
진짜 엔지니어죠.



맞아요! 내부 마법을 걷어내고
O(1)의 원리를 뼈에 새기기 위해
djb2 고분산 해시 함수까지
직접 짜본 거잖아요.

목표: 문자열 키를 O(1) 평균 시간에 조회하고,
로드 팩터 0.75 도달 시 2배 크기로 재해싱(Rehashing) 구현.

Gemini Notebook

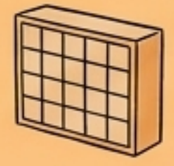
제이의 시각적 해설: 16개 우편함과 서류철



1. 해시 함수 (티켓 발권기)
2. 우편함 배정 (충돌 & 체이닝)
3. 동적 확장 (리해싱, Rehashing)



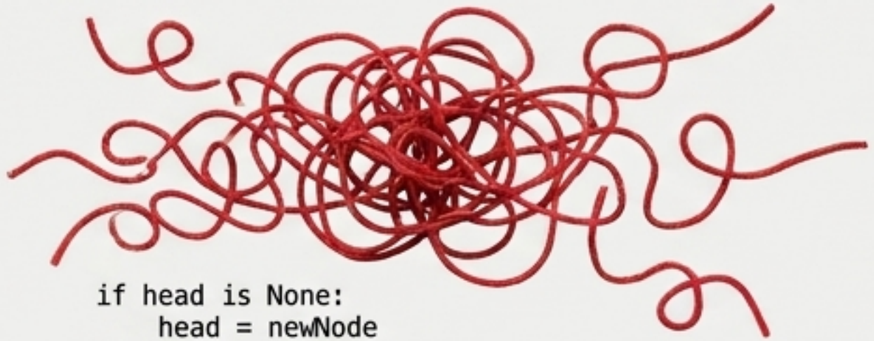
주민번호를 수학 공식(djb2)에 넣으면 딱 1개의 번호표가 나옵니다.
1초 만에 해당 우편함으로 달려가면 끝이죠.
서류가 75% 차서 복잡해지면? 관리소장이 우편함을 2배로 늘리고
서류를 널찍하게 다시 분배(Rehashing)합니다!



Gemini Notebook

토론 2. 앞뒤 빈 껍데기 '더미 노드', 과연 메모리 낭비일까?

Alex' voice: 노드가 0개이거나 1개일 때 발생하는 Head == None 같은 수많은 엣지 케이스를 어떻게 방어할 겁니까?



```
if head is None:
    head = newNode
elif head.next is None:
    if newNode.val < head.val:
        newNode.next = head
        head = newNode
    else:
        head.next = newNode
else:
    # More complex logic...
    current = head
    while current.next is not None:
        # ...
```

센티넬 노드가 없을 때: 수많은 if-else 분기문 가득. 버그 발생률 폭발.




```
newNode.prev = tail.prev
newNode.next = tail
tail.prev.next = newNode
tail.prev = newNode
```

센티넬 노드가 있을 때: 항상 존재하는 Dummy Head/Tail 덕분에 분기문 0개. 단 4줄의 코드로 완전한 O(1) 원자성 보장.

Gemini Notebook

제이의 시각적 해설: 양손을 꽉 잡은 기차 놀이



1. 기차 놀이 대열 (모든 친구가 손을 잡고 있음)



2. 중간에서 이탈 (손을 놓음)



3. 양옆 친구들 손잡기 (즉시 연결)



4. 맨 앞으로 순간이동 (새치기 완료)



10억 명이 줄을 서 있어도, **중간에 한 명이 빠져나와 맨 앞으로 새치기할 때 반 전체가 다시 줄을 설 필요가 없습니다.**
색인 장부(HashMap)로 위치를 찾고, 단 4번만 앞뒤 친구와 손(Pointer)을 바꿔 잡으면 끝나는 O(1) 순간이동의 마법이죠.



Gemini Notebook

토론 3. 매초 1,000만 개의 TTL을 검사하면 서버가 뻗지 않을까?

Junior Mentor (Jay)

모든 키의 유통기한을 매초 다 뒤져보는 건
불가능해요. 최소 힙(Min-Heap)은
골루 있습하니 어떻게 이 병목을 박살 냈죠?



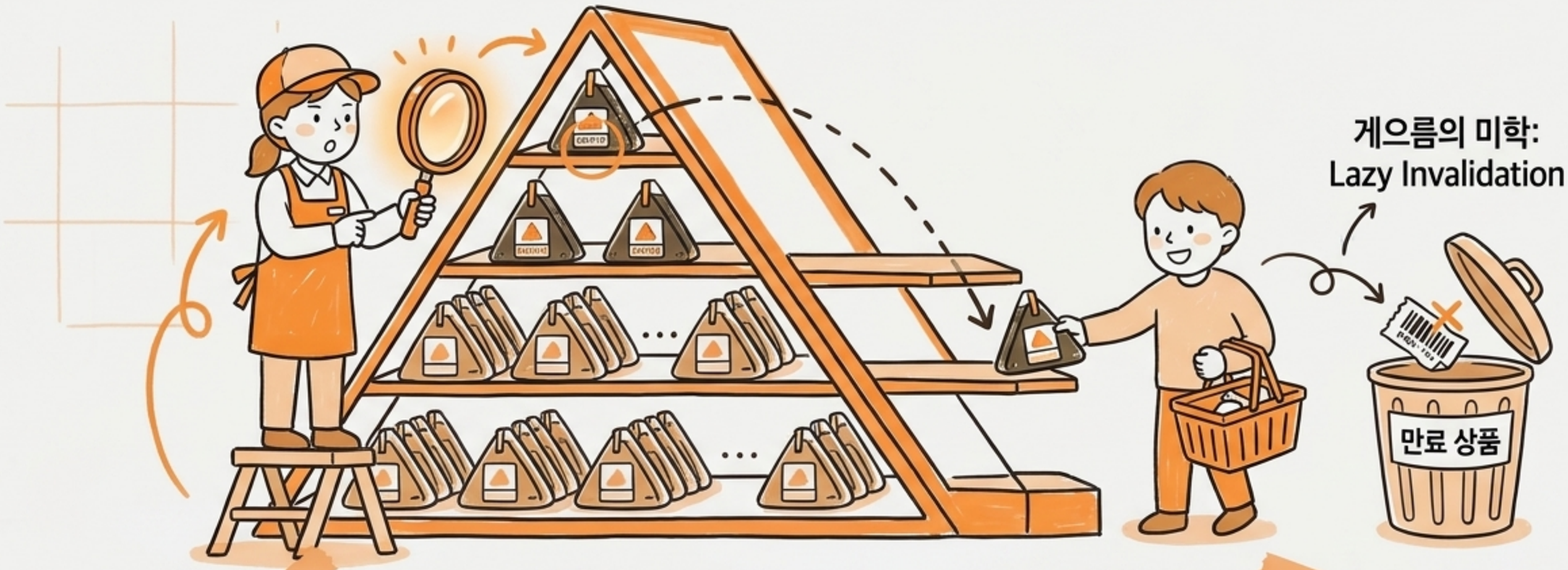
Senior Architect (Alex)

수만 개의 키를 선형 탐색 O(N)하는 대신,
만료 시각(expire_at)을 기준으로 0-인덱스 기반
완전 이진 트리(Min-Heap)를 구성합니다.
루트 노드 하나만 확인하면 O(1)로 끝납니다.



Gemini Notebook

제이의 시각적 해설: 편의점 유통기한 진열대





진열대 꼭대기 김밥 1개의 유통기한만 슬쩍 보고 안전하면 나머지 999만 개는 볼 필요도 없습니다! 그리고 고객이 새 상품으로 덮어써서 과거 데이터가 쓰레기가 되면, 굳이 진열대를 뒤집어 찾지 않고 나중에 꼭대기로 울라올 때 장부와 비교해 버리는 '게으름의 미학(Lazy Invalidation)'을 씁니다.

Gemini Notebook

보이지 않는 엣지 케이스: 메모리는 글자 수가 아니다



Windows 콘솔의 한글 인코딩 함정에 빠지면 안 됩니다.
진정한 인메모리 관리자(MemoryManager)는 len("홍길동")인 3이 아니라,
UTF-8 인코딩된 정밀한 9바이트 단위로 회계를 추적합니다.

Gemini Notebook

Mini Redis 아키텍처 해독 매트릭스

해결한 엔지니어링 문제	적용된 기술 (Alex)	직관적 멘탈 모델 (Jay)
조회/저장 문제 (Collision)	체이닝 해시맵 & djb2 리사이징	16개 우편함과 서류철 이사
메모리 방출 (LRU Eviction)	이중 연결 리스트 + 해시맵 O(1) 이동 O(1)	양손 맞잡은 기차 놀이와 색인 장부
만료 처리 (TTL/Lazy Deletion)	최소 힙 O(1) Peek & 지연 무효화	편의점 진열대와 게으른 알바생

Gemini Notebook

주니어 개발자가 이 코드를 통해 뼈에 새겨야 할 3가지

1. O(1) 순간이동의 마법

포인터 4개만 정확히 다루면 10억 개의 배열 복사 비용을 아낄 수 있다.

2. 게으름의 미학 (Lazy Deletion)

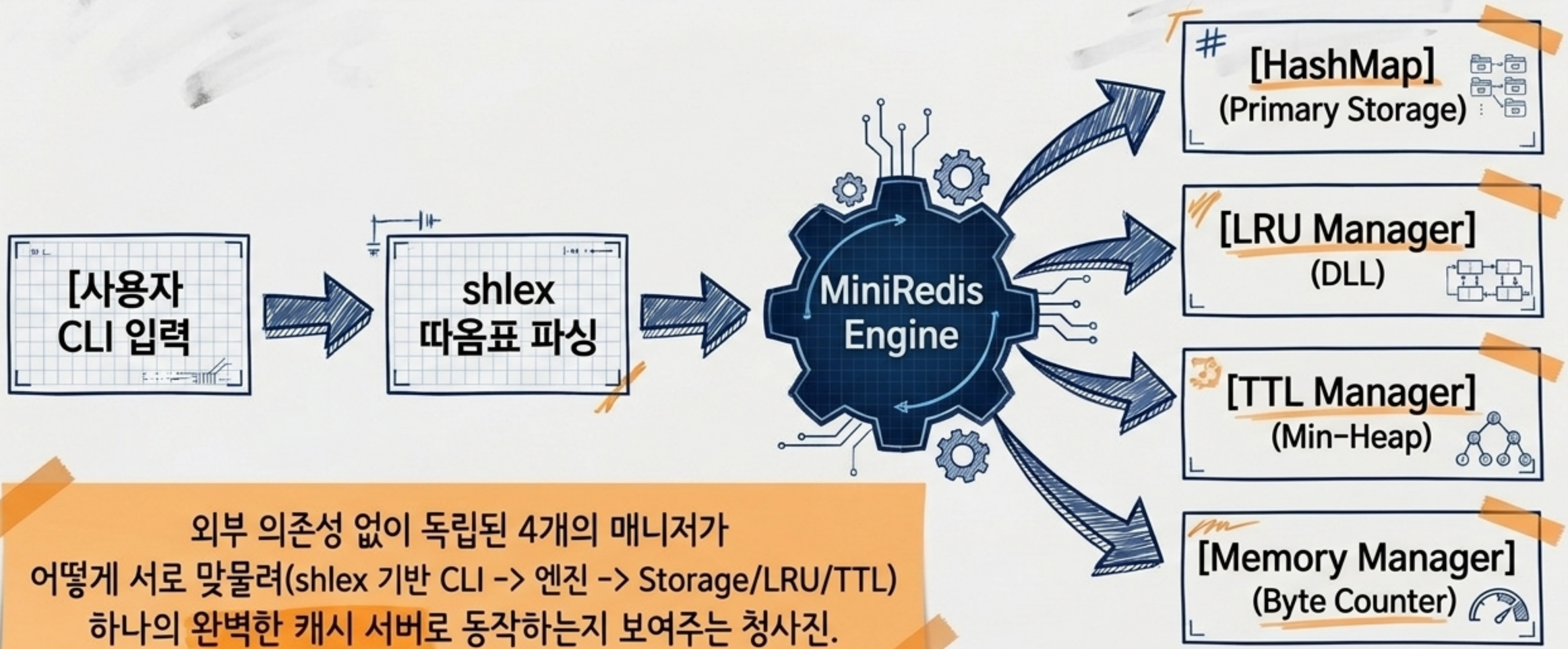
완벽한 실시간 동기화보다 필요할 때 확인하고 버리는 것이 진짜 최적화된 시스템이다.

3. 정밀한 메모리 회계

시스템 엔지니링에서는 문자열의 길이가 아닌 'UTF-8 바이트' 단위로 사고해야 한다.

Gemini Notebook

전체 시스템의 유기적 협력 (Architecture Flow)

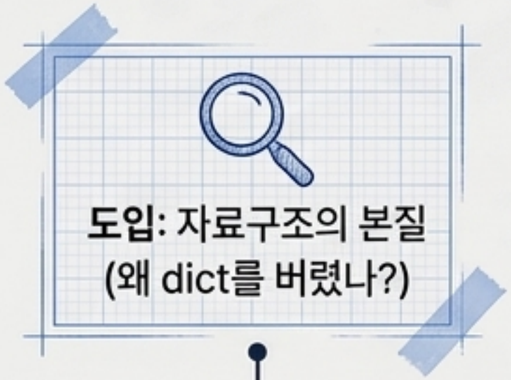


외부 의존성 없이 독립된 4개의 매니저가
어떻게 서로 맞물려(shlex 기반 CLI -> 엔진 -> Storage/LRU/TTL)
하나의 완벽한 캐시 서버로 동작하는지 보여주는 청사진.

Gemini Notebook

REC

Audio Overview 기획안: 에피소드 진행 흐름도



도입: 자료구조의 본질
(왜 dict를 버렸나?)



1부: 체이닝 해시맵과
동적 리사이징



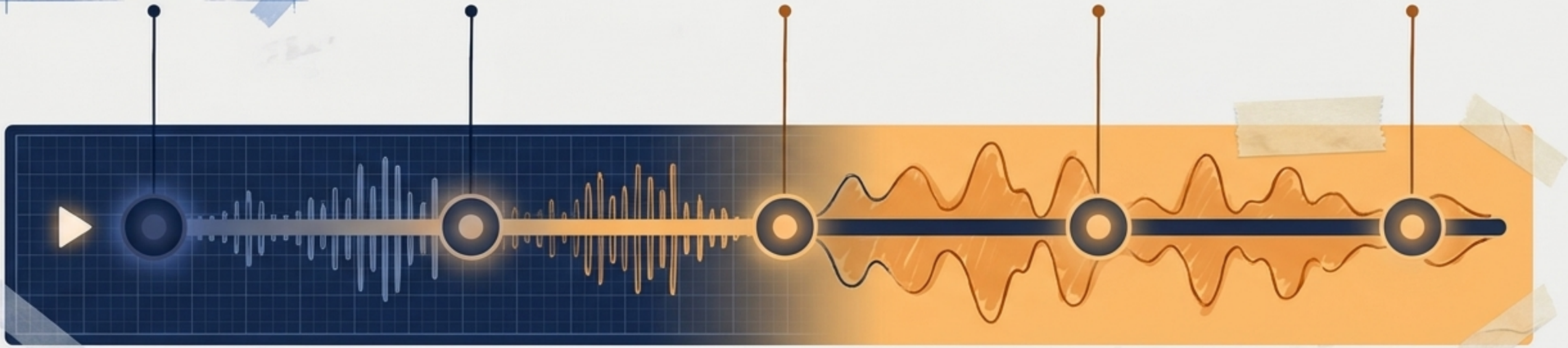
2부: 기차 대형으로 푸는
O(1) LRU 방출

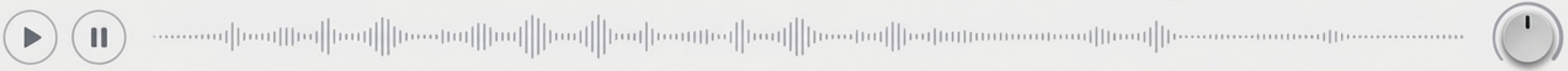


3부: 편의점 비유로 푸는
Min-Heap TTL 지혜



마무리: 주니어가 얻어가야 할
시니어급 사고방식





Gemini Notebook

나만의 아키텍처 팟캐스트 생성하기 (NotebookLM Prompt)



Copy

```
당신들은 세계 최고의 IT 팟캐스트 진행자 듀오인
시니어 아키텍트 '알렉스'와 주니어 멘토 '제이'입니다.

제공된 소스 코드를 바탕으로 Mini Redis 프로젝트에 대해
깊이 있는 기술 대담을 나누어주세요.

복잡한 문법 나열은 지양하고, 재치 있는 티키타카와
직관적인 실생활 비유(우편함, 기차놀이, 편의점 진열대)를
중심으로 청각적 몰입감을 극대화하세요.
```



내장 라이브러리의 안락함을 벗어날 때, 진짜 엔지니어링이 시작됩니다.

(전체 소스코드를 NotebookLM에 넣고 이 프롬프트를 적용해보세요)

Gemini Notebook